Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled "Core JDO" are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdobook@yahoogr oups.com

CHAPTER 8

Comment: This chapter has been moved to number 9 in an updated TOC. The numbering below reflects that.

"Creativity is the power to connect the seemingly unconnected"- William Plomer

JDO and the J2EE Connector Architecture

This chapter explains how JDO can be used in conjunction the Java 2 Platform, Enterprise Edition (J2EE) via the J2EE Connector Architecture (JCA). This allows a JDO implementation to collaborate with a J2EE compliant Application Server on security, transaction and connection management. This chapter assumes familiarity with the J2EE concepts and development.

This chapter covers:

- A brief introduction to JCA
- How JCA and JDO fit together
- Using JDO and JCA to build J2EE applications
- Using JDO without JCA to build J2EE applications

Chapter 10 provides greater detail on actually using JDO to build J2EE applications. This chapter aims to provide an introduction by explaining how JDO and J2EE can interoperate. In this context, a J2EE application refers to either a Servlet or an Enterprise JavaBean (EJB).

9.1 JCA Overview

The JCA specification, part of the overall J2EE specification, is intended as a standard mechanism for plugging heterogeneous Enterprise Information Systems into a J2EE compliant Application Server. It allows J2EE applications to securely interact with non-J2EE applications in a transactional manner. Providing there is a JCA resource adapter for the Enter-

prise Information System (EIS), the EIS and Application Server can collaborate on security, transaction and connection management. A resource adapter is standard, system-level software driver for the EIS.

The JCA specification (version 1.0) defines three system-level contracts:

Connection Management Contract this allows the J2EE platform to pool connections to the EIS and allow applications to use those connections to interact with the EIS when needed.

Transaction Management Contract this allows the J2EE platform to control interaction with the EIS transactionally.

Security Contract this allows the J2EE platform to secure access to the EIS.

These contracts allow the Application Server to manage interaction with the EIS in a standard way. However, they are system-level contracts and as such are of concern only to those developing JCA resource adapters.

The Latest version of the JCA specification (version 1.5) has added additional system-level contracts, however these are of little relevance when using JDO within the J2EE environment at this time.

Figure 91 depicts how a resource adapter plugs into an Application Server and through the JCA system-level contracts manages the interaction between the Application Server and the EIS. As can be seen, the JCA resource adapter runs within the same JVM as the Application Server:

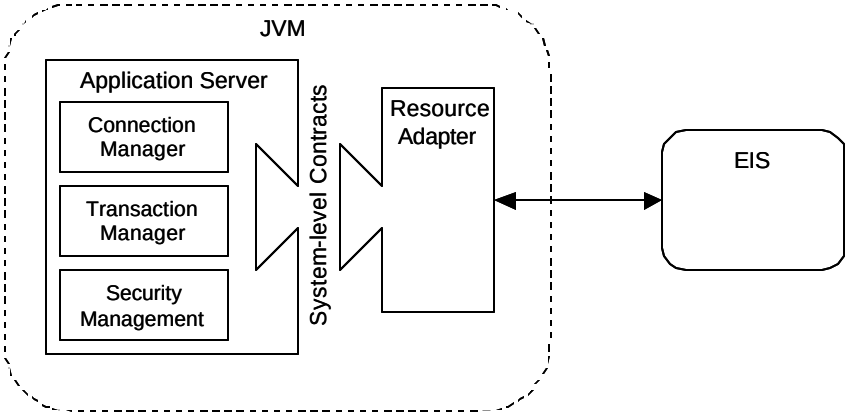

Figure 9-1 JCA Architecture

The JCA specification also defines a Common Client Interface (CCI). The CCI defines standard APIs for J2EE applications to connect and interact with an EIS in a standardized, record-oriented manner.

9.2 JDO AND JCA

Although JCA was principally designed as a standard mechanism to plug an EIS into an Application Server, it also provides a convenient mechanism for a JDO implementation to do likewise. As such, any JDO implementation designed to work within the J2EE environment will likely provide a JCA resource adapter. This resource adapter means J2EE applications can use JDO, since the JDO implementation will collaborate with the Application Server on security, connection and transaction management as defined by JCA.

In the same manner that a J2EE application can use JDBC today and benefit from container managed connections and transactions, via JCA an application using JDO gets the same benefits. So even though JDO is not officially part of the J2EE specification, it can still leverage the enterprise services offered by the J2EE environment.

Support for JCA

The JDO specification does not actually require a JDO implementation to provide a JCA resource adapter. Therefore a developer wishing to use JDO within the J2EE environment should ensure the chosen JDO implementation does indeed support JCA.

A JDO implementation is primarily interested in implementing the JCA system-level contracts, since these allow it to collaborate with an Application Server. The JCA CCI is not of particular relevance to JDO, since JDO is itself the "client" API. The only CCI APIs of relevance to JDO are those involved with connections. The CCI defines two interfaces related to connection management, ConnectionFactory and Connection (which will be covered later in this chapter).

9.3 USING JDO AND JCA

This section goes through how to use JDO to build a J2EE application. It covers:

- Setup and Configuration – setting up and configuring a JCA resource adapter
- Connection management – using the PersistenceManagerFactory, PersistenceManager interfaces and the role of the JCA ConnectionFactory and Connection interfaces
- Transaction management – bean versus container managed transactions and distributed transactions
- Security management

9.3.1 Setup and Configuration

The first step in using a JDO implementation with an Application Server is to setup and configure an instance of the JCA resource adapter provided by the JDO implementation.

The resource adapter itself is contained in a resource adapter archive or RAR file. The RAR file contains, amongst other things, the classes used to implement the JCA system-level contracts and a deployment descriptor (called "ra.xml"). The deployment descriptor defines the classes that implement the JCA system-level contracts along with the configuration properties that are specified at deployment time. These configuration properties are akin to the properties used when creating a PersistenceManagerFactory instance (ConnectionURL; UserName; Password).

Different Application Servers provide different mechanisms for deploying an instance of a resource adapter. Some simply require the RAR file to be copied into a particular directory along with an XML configuration file containing, amongst other things, the configuration properties for the resource adapter instance. The deployment process results in a PersistenceManagerFactory instance being bound to a specified JNDI name. This name can then be used by a J2EE application to retrieve the PersistenceManagerFactory instance.

The PersistenceManagerFactory instance created during the deployment of the resource adapter automatically delegates connection pooling to the Application Server. It also

ensures the PersistenceManager instances are automatically enlisted in any transaction, as appropriate. The PersistenceManager instances delegate transaction management to the Application Server and any attempt to explicitly start or end a transaction using JDO's Transaction interface will result in JDOUserException being thrown.

9.3.2 Connection Management

The JCA connection management contract enables a JDO implementation to delegate connection management to an Application Server. This allows a JDO implementation to take advantage of the pooling features and quality of service guarantees offered by the Application Server. In addition it means that administrators can use the interfaces provided by the Application Server to configure and manage all connection pools, JDO or otherwise.

9.3.2.1 Getting a PersistenceManagerFactory

Once an instance of the resource adapter has been deployed a J2EE application can use JNDI to lookup the PersistenceManagerFactory instance.

This lookup would typically be done during initialization. For a Servlet this would be done in its init() method, for an EJB it would be in its setSessionContext() method. In both cases an instance field defined by the class would be used to maintain a reference to the PersistenceManagerFactory instance so that the Servlet or EJB methods can use it to get PersistenceManager instances when needed.

The following code snippet shows how an EJB would get a PersistenceManagerFactory instance. It assumes that an instance of a resource adapter was deployed using the JNDI name "jdo/example":

```
private PersistenceManagerFactory pmf;

public void setSessionContext(SessionContext ctx) {

  pmf = (PersistenceManagerFactory)
    new InitialContext().lookup(
      "java:comp/env/jdo/example");
}
```

The following code snippet shows how a Servlet would do likewise:

```
private PersistenceManagerFactory pmf;

public void init(ServletConfig config) {

  pmf = (PersistenceManagerFactory)
```

```
      new InitialContext().lookup(
        "java:comp/env/jdo/example");
  }
```

Standalone Servlet Engines

JDO can also be used with standalone Servlet engines that do not support JCA. In this situation the init() method simply creates a PersistenceManagerFactory instance as per any unmanaged JDO application.

9.3.2.2 Getting a PersistenceManager

Once a PersistenceManagerFactory instance has been created a Servlet or EJB can use it to get and close PersistenceManager instances as per normal. For an EJB, each bean method would need to get a PersistenceManager instance before it does anything, for a servlet the doGet() and doPost() methods would do likewise.

9.3.2.3 ConnectionFactory and Connection Interfaces

Since the JDO specification doesn't require a JDO implementation to support JCA or define how JCA should be supported there is the possibility of variance as to exactly how a JDO implementation supports JCA.

JCA CCI defines two interfaces related to connection management: ConnectionFactory and Connection. These are equivalent in purpose to JDO's PersistenceManagerFactory and PersistenceManager interfaces. A JDO implementation may choose to use the CCI interfaces as the means of getting PersistenceManager instances. In this case rather than looking up a PersistenceManagerFactory instance via JNDI, a ConnectionFactory instance is looked up instead. Using the getConnection() method defined on ConnectionFactory, the J2EE application would get a Connection instance. Depending on the JDO implementation it might be possible to cast the Connection instance to a PersistenceManager directly or else use an implementation specific API to retrieve the PersistenceManager instance from the Connection instance.

Some JDO implementations may support the use of the CCI interfaces completely interchangeably with the JDO PersistenceManagerFactory and PersistenceManager interfaces. In this case the implementation's PersistenceManagerFactory and PersistenceManager classes also implement the CCI ConnectionFactory and Connection interfaces. Depending on the application's perspective it can decide to be either JCA or JDO compliant in the way it gets its connections.

9.3.3 Transaction Management

The JCA transaction management contract enables a JDO implementation to delegate transaction control to an Application Server.

With Servlets, only explicit transaction control is possible. However, when using JDO to develop an EJB it is possible to either explicitly control transactions (bean-managed transactions) or delegate transaction control to the EJB container (container-managed transactions).

When using JDO with Servlets, the doGet() and doPost() methods would get a PersistenceManager instance, begin a transaction, do something, end the transaction and close the PersistenceManager instance.

For an EJB using bean-managed transactions, each bean method would do likewise. However for an EJB using container-managed transactions the EJB container implicitly begins and ends transactions. In this case each bean method simply gets a PersistenceManager instance, does something interesting and closes it.

Handling Exceptions

When using JDO with Servlets and EJBs care needs to be taken to ensure that the PersistenceManager instance is always closed when finished with, even in the event of an exception. Since Servlets and EJBs are constantly running, if a PersistenceManager instance is not closed then it cannot be reused which results in a resource leak.

See Chapter 4 for more details on handling exceptions properly.

The benefit of container-managed transactions is that a transaction can span multiple bean method invocations.

9.3.3.1 Bean-managed Transactions

With bean-managed transactions each time getPersistenceManager() is called within a bean method it will return a different PersistenceManager instance. If one bean method calls another, each will therefore get its own PersistenceManager instance. This means each bean method operates within its own local transaction.

It is possible to devise a simple framework that allows different bean method calls to share the same PersistenceManager instance. However, a better approach is to use container-managed transactions instead.

9.3.3.2 Container-managed Transactions

With container-managed transactions multiple bean methods calls can be made within the same transaction (based on the transaction attribute declared for the bean methods). In this case, each time getPersistenceManager() is called it returns the PersistenceManager instance

enlisted with the current transaction. This way multiple bean methods share the same PersistenceManager instance.

Remote Invocation

Since EJBs have remote interfaces it is possible that one bean can call a method on another in a different JVM. In this situation the bean methods cannot possibly share the same PersistenceManager instance. Instead a PersistenceManager instance is associated with the transaction within each JVM and the Java Transaction Service (JTS) is responsible for coordinating them all as a single, distributed transaction.

9.3.3.3 Distributed Transactions

Usually a transactional resource, like an RDBMS, manages its own transactions. When an application commits, the RDBMS itself determines if the transaction is successful or not.

However, if an application needs to interact with several different systems but still requires transactional guarantees across all those systems, then it needs to use an external transaction manager. The external transaction manager is responsible for coordinating access to each system as a single transaction – either all systems will be updated or none of them will.

These distributed transactions are of primary use when there is a requirement to coordinate access across different systems. For example an application may need to read a message from a queue and update a relational database as a single operation (to ensure the message gets written into the database). In this situation a distributed transaction can be used to coordinate the message queue and the database.

The JCA specification defines three classifications for resource adapters, based on their level of transaction support:

NoTransaction the resource adapter does not support transactions. Each interaction with the EIS is an atomic operation in itself.

LocalTransaction the resource adapter supports local transactions only. Multiple interactions to the same EIS can be coordinated transactionally, but it cannot be used as part of a distributed transaction.

XATransaction the resource adapter supports local and distributed transactions.

Some JDO implementations provide LocalTransaction resource adapters only. This means that a bean method using JDO cannot be coordinated transactionally with another bean method, unless both methods are invoked within the same JVM and both use the same PersistenceManagerFactory instance (and therefore each method gets the same PersistenceManager instance).

If an application needs to coordinate the following transactionally:

- Bean methods across multiple JVMs
- Bean methods that use different PersistenceManagerFactory instances
- Bean methods accessing other transactional systems (EIS; RDBMS)

Then the chosen JDO implementation needs to provide a XATransaction capable resource adapter.

9.3.4 Security

Since JDO does not define its own security model and instead relies on the security provided by the underlying datastore, the JCA security contract is of less relevance to JDO than the connection and transaction contracts.

The JDO implementation's resource adapter will provide an appropriate implementation of the security contract given the underlying datastore being used. In most cases this relies on a traditional username and password being specified when the resource adapter is deployed.

9.5 USING JDO WITHOUT JCA

As has already been outlined, JDO can be used with Servlets without needing to use a JCA resource adapter. In this case the Servlet is the same as any other unmanaged JDO application. Likewise, if an EJB does not require container-managed transactions then it is not necessary to use a JCA resource adapter either.

Depending on needs, the Servlet or EJB can rely on the connection pooling provided by JDO implementation's PersistenceManagerFactory or instead configure an external connection factory using the ConnectionFactory or ConnectionFactoryName PersistenceManagerFactory properties. See Chapter 5 for more details on creating a PersistenceManagerFactory instance in this way.

9.6 SUMMARY

This chapter has provided an introduction to using JDO to build J2EE applications by examining how JDO and J2EE compliant Application Servers can interoperate via JCA.

The next chapter will go into greater detail on actually developing a J2EE application using JDO.